

ConfuGuard: Using Metadata to Detect Active and Stealthy Package Confusion Attacks Accurately and at Scale

Wenxin Jiang*
Purdue University
West Lafayette, IN, USA
Socket
Wilmington, DE, USA
jiang784@purdue.edu

Berk Çakar
Purdue University
West Lafayette, IN, USA
bcakar@purdue.edu

Mikola Lysenko
Socket
Wilmington, DE, USA
mik@socket.dev

James C Davis
Purdue University
West Lafayette, IN, USA
davisjam@purdue.edu

Abstract

Package confusion attacks, such as typosquatting, threaten software supply chains. Attackers make packages with names that syntactically or semantically resemble legitimate ones, tricking engineers into installing malware. While prior work has developed defenses against package confusions in some software package registries, notably NPM, PyPI, and RubyGems, gaps remain: high false-positive rates, generalization to more software package ecosystems, and insights from real-world deployment.

In this work, we introduce ConfuGuard, a state-of-the-art detector for package confusion threats. We begin by presenting the first empirical analysis of benign signals derived from prior package confusion data, uncovering their threat patterns, engineering practices, and measurable attributes. Advancing existing detectors, we leverage package metadata to distinguish benign packages, and extend support from three up to seven software package registries. Our approach significantly reduces false positive rates (from 80% to 28%), at the cost of an additional 14s average latency to filter out benign packages by analyzing the package metadata. ConfuGuard is used in production at our industry partner, whose analysts have already confirmed 630 real attacks detected by ConfuGuard.

CCS Concepts

• **Security and privacy** → **Software security engineering; Malware and its mitigation**; • **Software and its engineering** → **Software libraries and repositories**.

Keywords

Software supply chain security, typosquatting, package confusion attacks, malware detection, software package registries

ACM Reference Format:

Wenxin Jiang, Berk Çakar, Mikola Lysenko, and James C Davis. 2026. ConfuGuard: Using Metadata to Detect Active and Stealthy Package Confusion Attacks Accurately and at Scale. In *2026 IEEE/ACM 48th International Conference on Software Engineering (ICSE '26)*, April 12–18, 2026, Rio de Janeiro, Brazil. ACM, New York, NY, USA, 13 pages. <https://doi.org/10.1145/3744916.3773229>

*Most of this work was conducted during the author's internship at Socket.



This work is licensed under a Creative Commons Attribution 4.0 International License. *ICSE '26, Rio de Janeiro, Brazil*

© 2026 Copyright held by the owner/author(s).

ACM ISBN 979-8-4007-2025-3/26/04

<https://doi.org/10.1145/3744916.3773229>

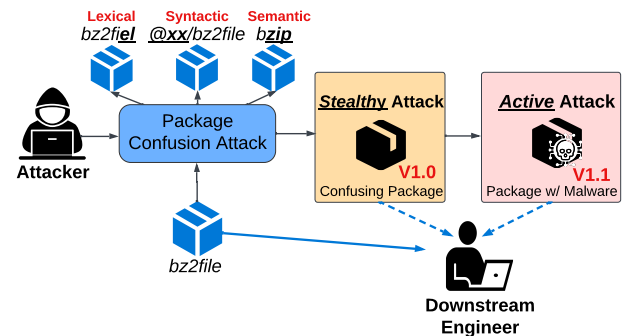


Figure 1: Threat model illustrating common active and stealthy package confusion attacks. Attackers spoof a legitimate package such as `bz2file` with lexical typos (`bz2fiel`), syntactic namespace manipulations in scoped registries (`@xx/bz2file`), and semantic substitutions (`bzip`). Attackers release these look-alike packages as *active* attacks (malicious from the first version) or *stealthy* attacks that are benign until a later update injects malware. Engineers may be tricked into depending on the impostor directly or as a transitive dependency.

1 Introduction

Software package registries (SPRs) are indispensable in modern development. They help engineers develop applications by integrating open-source packages, reducing costs and accelerating product cycles [56, 63, 77]. SPRs are used by industry and governments [1, 67], including in AI and safety-critical applications [30, 65]. The resulting software supply chain is an attractive target—if an adversary can cause an engineer to install a malicious package, the attacker may gain access to downstream users' resources. One of their tactics is *package confusion attacks* [33, 49]: creating malicious packages with misleading names [26, 36, 52]. Figure 1 shows the attack model, covering naming tactics (lexical, syntactic, and semantic confusion) and attack timing (active vs. stealthy).

Researchers have proposed detectors to mitigate package confusion attacks. Early solutions detected packages with lexically similar names [61, 68, 75]. State-of-the-art approaches have begun exploring semantic similarity [49]. However, as criticized by Ohm *et al.* [51], prior work did not emphasize low false positive rates, which are needed for real-world use [51]. We note also that prior work focuses on limited SPRs, notably NPM and PyPI; and that the literature lacks experience reports from production deployment.

In this study we propose a novel system to detect package confusion attacks. To reduce false positive rates, we examine the practices

of package confusion attackers and propose signals, based on package metadata, to distinguish benign packages from genuine (though perhaps stealthy) threats. To generalize prior work to more SPRs, we analyze naming practices and confusion attacks in seven SPRs, and identify novel aspects of these SPRs that expand the attack surface for package confusion. We also describe an embedding-based name comparison method and a novel nearest-neighbor search algorithm that improve the accuracy and efficiency of detection.

We integrated these techniques in the ConFuGuard system. We initially evaluated ConFuGuard on previous package confusion attack datasets (§4), and iteratively enhanced it using recent real-world package confusion reports flagged and triaged by our industry partner’s security analysts. We report that ConFuGuard achieves a false positive rate of 28%, an improvement of 52% over the state of the art. ConFuGuard has average latency of 31 seconds per package on a modern server (§6.6). Our industry partner has already used ConFuGuard in production to identify 630 confirmed package confusion attacks.

To summarize our contributions:

- We refine the definition of package confusion attacks to encompass both active and stealthy threats.
- We generalize prior analyses to several new SPRs, noting the effect of package naming hierarchies on confusion attack tactics.
- We propose novel algorithms to improve the accuracy and efficiency of detecting package confusion attacks.
- We open-source ConFuDB, our production dataset containing 2,361 package confusion threats triaged by security analysts.
- We share lessons learned from using ConFuGuard in production.

Significance: Package confusion attacks are common yet challenging to detect. This paper introduces ConFuGuard, a best-in-class detection system for package confusion detection, designed through analysis of previous attacks and insights from ongoing deployment. ConFuGuard is used in production by our industry partner.

2 Background and Related Work

Here, we provide background on SPRs and their package naming conventions (§2.1), review the taxonomy of package confusion attacks (§2.2), and outline existing defense mechanisms (§2.3).

2.1 SPRs and Package Naming

Much of modern software development depends on community ecosystems of libraries and frameworks that facilitate software reuse [77, 79]. We refer to these reusable software artifacts as *software packages*. These packages are distributed by *software package registries (SPRs)* [59, 63]. Table 1 shows the SPRs for seven popular ecosystems comprising over 8 million software packages. Some SPRs restrict package publication to individual accounts, so the publisher’s identifier does not appear in the package name. Others allow grouping packages into namespaces or organizations, which are incorporated into the package name. We call the former package naming structure “flat” and the latter “hierarchical” [6].

SPRs typically act as infrastructure and leave it to the engineering community to establish norms for interaction. One aspect of their “hands-off” approach is that SPRs impose almost no constraints on package names [3, 13, 16, 17, 22, 42]. This flexibility introduces two related challenges for engineers: First, based on the application

Table 1: SPRs analyzed in this study, including their programming ecosystem, package naming structure, and presence in prior work on package confusion detection. Registry sizes are as of Jan. 2025. An example flat naming pattern is PyPI’s requests module. An example hierarchical name is Hugging Face’s google-bert/bert-base-uncased.

Registry (# pkgs)	Ecosystem	Name Structure	Prior Work
PyPI (619K)	Python	Flat	[49, 75]
RubyGems (212K)	Ruby	Flat	[49]
Maven (503K)	Java	Hierarchical	N/A
Golang (175K)	Go	Hierarchical	N/A
Hugging Face (1.1M)	AI Models	Hierarchical	[29]
NPM (5.1M)	JavaScript	Both	[49, 68]
NuGet (430K)	.NET	Both	N/A

domain (e.g., traditional software libraries vs. machine learning model packages), engineers across different SPRs have adopted different naming conventions [14, 29], which may also confuse newcomers. Second, as we detail next, SPRs’ permissive naming practices enable deliberate confusion attacks.

2.2 Package Confusion Attacks and Taxonomy

Package confusion is an attack in which adversaries seek to persuade engineers to install malicious packages [49].¹ Attackers typically choose a package name similar to a legitimate one that meets the engineer’s needs [49]. Attacks may be multi-pronged, e.g., using advertising campaigns to increase their chances of fooling engineers [28, 62]. Package confusion attacks are an ongoing concern, with hundreds of packages detected by researchers in Socket [69], ReversingLabs [58], Orca [12], and others [7].

Table 2 summarizes the state-of-the-art taxonomy of package confusion techniques, based on Neupane *et al.* [49]. The variations in naming structure in different SPRs also affect the nature of package confusion by SPR. In SPRs with a flat naming structure, confusion attacks look like the top portion of the table. In hierarchical SPRs, attackers have a larger naming surface to exploit, as illustrated in the bottom portion of the table.

Table 2: Common package confusion techniques with examples. The top section presents some of the taxonomy proposed by Neupane *et al.* [49]. We extend this taxonomy by adding three new patterns (bottom; see §4.1) that generalize confusion strategies observed in additional SPRs. The examples for those new patterns are drawn from package confusion threats flagged by ConFuGuard.

Technique	Example (<i>basis</i> → <i>attack</i>)
1-step Levenshtein Distance	crypto → crypt
Sequence reordering	python-nmap → <u>nmap-python</u>
Scope confusion	@cicada/render → cicada-render
Semantic substitution	<u>bz2file</u> → <u>bzip</u>
Alternate spelling	colorama → colourama
Impersonation Squatting	<u>meta-llama/Llama-2-7b-chat-hf</u> → <u>facebook-llama/Llama-2-7b-chat-hf</u>
Compound Squatting	@typescript-eslint/eslint- <u>plugin</u> → @typescript- <u>eslint</u> / <u>eslint</u>
Domain Confusion (Golang)	<u>github.com/prometheus/prometheus</u> → <u>git.luolix.top/prometheus/prometheus</u>

¹Package confusion is the SPR-specific manifestation of the general naming confusion attack, which affects all IT endeavors including DNS (domain names) [18, 23, 72], BNS (Blockchain names) [47], and container images (image names) [5, 40].

2.3 Defenses Against Package Confusion

Package confusion attacks can be mitigated at many points in the reuse process, *e.g.*, by scanning for malware in registries [74] or during dependency installation [37] or by sandboxing dependencies at runtime [4, 21, 73]. Such techniques provide good security properties, but are computationally expensive. Our approach is part of a class of cheaper techniques that leverage the most pertinent attribute of a package confusion attack: the similarity of the name to that of another package. We describe the literature in this vein.

2.3.1 Approaches. In the academic literature, the earliest name-oriented defenses were purely lexical: Taylor *et al.* [68] and Vu *et al.* [75] measured similarity using Levenshtein distance [38] to identify PyPI packages with minor textual alterations from one another. To detect confusion attacks based on name semantics (*e.g.*, `bz2file` vs. `bzip`), Neupane *et al.* [49] applied FastText embeddings [9] to enable an abstract comparison of package names.

Recognizing the threat of package confusion attacks, industry has also offered some defenses. NPM and PyPI both use lexical distances to flag packages or otherwise reduce user errors [50, 55, 80]. Some other companies also offer relevant security tools. For example, Stacklok [64] and Microsoft’s OSSGadget [46] can both detect potential package confusion attacks.

Except for Stacklok, which uses author activity metrics, these defenses rely solely on the characters in the package name, whether through edit-distance heuristics or semantic similarity. We refer to these lexical or semantic name features collectively as “*string-only signals*”. These defenses ignore auxiliary metadata such as popularity, maintainers, and READMEs.

2.4 Gaps in Current Approaches

Despite the proposed approaches for detecting package confusion attacks, both theoretical and practical gaps remain.

2.4.1 Theoretical Gaps. Current detectors primarily depend on *string-only signals*, disregarding contextual metadata. This narrow view is likely to inflate false positive rates when legitimate projects share similar names, while allowing stealthy threats to go undetected when attackers use benign-looking package names combined with adversarially crafted metadata (*e.g.*, package descriptions, READMEs) to deceive engineers. Furthermore, prior work’s *definition of package confusion* requires packages to contain active malware [49], failing to account for stealthy threats that initially appear benign but may become malicious later. This oversight highlights a gap in understanding the full threat landscape.

Additionally, detailed empirical treatments have so far focused exclusively on package confusion attacks in NPM, PyPI, and RubyGems [49, 68, 75], omitting package confusion attacks in SPRs with *hierarchical naming structures* such as Maven, Golang, and Hugging Face. This limited scope means that existing detection algorithms and similarity metrics may not generalize to SPRs with hierarchical names, which offer attackers additional attack surfaces through namespace manipulation and compound squatting techniques.

2.4.2 Practical Gaps. The primary concern, as articulated by Ohm *et al.* [51], is that current package confusion detectors have *high false-positive rates*. Prior studies have shown that engineers consider monitoring systems with excessive false positives to be less useful

than conservative ones [20]. In the context of package confusion detection, false positives cause several practical issues: they lead to alert fatigue among security analysts; erode trust in the detection system; and harm developers’ reputations [20].

The engineering community acknowledges the severity of false positives. For example, developers of Microsoft’s OSSGadget typosquatting detection tool explicitly seek ways to reduce false positives [60]. Additionally, while industry reports mention package confusion defenses being used in production (*e.g.*, NPM and PyPI’s lexical distance checks, Stacklok’s risk scoring), these reports *lack concrete deployment details* such as false positive rates, detection accuracy, number of threats caught, or operational challenges. Without concrete performance metrics and deployment experiences, it remains unclear how effective these systems are in practice.

3 Problem Statement

In this section, we present an improved definition of package confusion threats (§3.1), outline our threat model (§3.2), and specify the system requirements necessary to effectively detect package confusion attacks (§3.3).

3.1 Refined Definition of Package Confusion

We refine the definition of package confusion threat by emphasizing the manifestation of malicious intent. Our refined definition is supported by our analysis of package confusion attacks (§4.1).

Definition: Package Confusion Threat

A package confusion threat is a software package whose name mimics a trusted resource, with the intent of deceiving developers into installing code that is actively malicious or may become so (stealthily).²

As depicted in Figure 1, the mimicry may be *lexical* (*i.e.*, typosquatting), *syntactic*, or *semantic*, but the intent is to confuse an engineer.

By framing our definition around package confusion *threats* rather than just *attacks*, we admit the possibility that packages may initially appear benign but harbor malicious intent. In contrast, prior work relies on malware detectors and only finds actively malicious packages [49, 68]. Expanding the package confusion detection scope from active attacks to stealthy threats enlarges the candidate set of similar package pairs and, if done naively, can inflate false positives. We mitigate this challenge through our empirical analysis of false positive patterns in existing package confusion datasets (§4.2), which reveals distinguishing signals between genuinely benign packages and stealthy threats awaiting activation.

3.2 Threat Model

We define the scope and boundaries of our threat model to clarify which package confusion attacks are addressed. Our model encompasses both the context in which these attacks occur and the specific threats we aim to detect.

Context Model: We focus on the SPRs mentioned in §2.1, including both traditional SPRs and emerging SPR for AI models.

²For comparison, Neupane *et al.* wrote: “Package confusion attack: a malicious package is created that is designed to be confused with a legitimate target package and downloaded by mistake” [49]. This requires the package to be actively malicious.

Threat Model: Our threat model distinguishes between package confusion attacks we address and those outside our scope:

- *In-scope:* We consider attacks where adversaries publish packages with names deceptively similar to legitimate packages within SPRs. Consistent with our refined definition, attackers may initially upload benign content (stealthy threats) and later inject malicious code (active threats).
- *Out-of-scope:* We exclude attacks that either use non-confusing package names or compromise existing legitimate packages through other means. Such attacks require different mitigation strategies [52].

This threat model is stronger than those of existing package confusion detection approaches [49, 68, 75] because we account for stealthy attacks that begin with non-malicious package content.

3.3 Requirements

A system for detecting package confusion threats must meet both security and operational goals. Working with our industry partner, we identified four requirements:

Req₁: High Precision and Recall. The system must balance precision (low false positive rate) against recall (low false negative rate). False negatives leave deceptive packages unflagged, but false positives lead to alert fatigue [2].

Req₂: Efficient and Timely Detection. The system must scale to large SPRs while making low-latency checks for real-time feedback.

Req₃: Compatibility Across Ecosystems. The system must support many SPRs with their diverse naming structures.

Req₄: High Recall of Stealthy Package Confusion Attacks. The system must identify both active and stealthy package confusions.

4 Analysis of Confusion Attacks

To address the high false positive rates of previous package confusion detection systems, Ohm *et al.* recently called for a deeper analysis of the available evidence [51]. We respond to their call in two ways: describing attackers' practices (§4.1), and reporting metadata features that distinguish true from false positives (§4.2).

4.1 Attackers' Practices

We examined first the role of stealth in confusion attacks, and then the attack variations in SPRs beyond NPM, PyPI, and RubyGems. Our findings informed [Req_{1,3,4}](#).

4.1.1 Stealthy Confusion Attacks. We analyzed 240 confirmed NPM package confusion "true positives" (*i.e.*, confusing packages that include malware), based on data provided by Neupane *et al.* [49]. We analyzed the number of days these malicious packages were available before malware was injected. We estimated the injection time based on the last version updated before removal by the SPR.

Figure 2 illustrates our findings on the release time before malware injection. Most package confusion attacks injected malware on the first day of release (*i.e.*, in the initial version). However, in 13.3% of the packages, attackers undertook stealthy attacks, deploying malware after days or weeks. Attackers may use this strategy to grow a user base before exploitation. Prior work on package confusion detection did not account for this behavior.

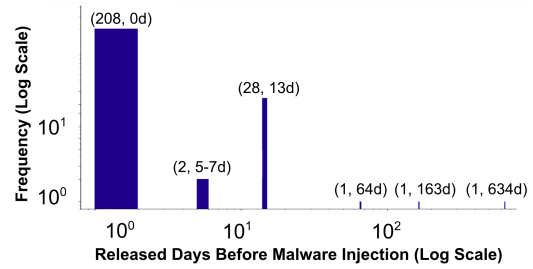


Figure 2: Distribution of time until malware, in packages with confusing names. 13.3% (32/240) of attacks occur ≥ 5 days after release.

4.1.2 Extending the Taxonomy to New SPRs. Prior work characterized malicious naming strategies in only a small set of SPRs, leaving gaps for ecosystems with hierarchical namespaces and platforms for hosting pre-trained AI models [34]. In collaboration with our industry partner's security analysts, who routinely triage package confusion incidents across all SPRs in Table 1, we reviewed Neupane *et al.*'s taxonomy against their field observations. Through this dialogue, we identified three additional mechanisms for package confusion attacks. Definitions follow; see Table 2 for examples.

- **Impersonation Squatting.** In hierarchical SPRs, attackers impersonate legitimate maintainers or organizations by registering a similar author or groupId.
- **Compound Squatting.** Making multiple coordinated edits to a hierarchical name, *e.g.*, altering both scope and delimiters at once. We consider this distinct because the compounded changes complicate conventional lexical comparisons.
- **Domain Confusion.** In SPRs where package names include URLs (*e.g.*, golang), attackers may register domains resembling those of trusted mirrors or proxies.

4.2 Metadata Features of Package Confusion

As discussed in §2.3, almost all package confusion detectors make use only of the names of packages. While this allows those systems to avoid costly content analysis, it leads to high false positive rates. We propose to integrate metadata to distinguish true from false positives. Although metadata signals can be circumvented, our approach aims to raise the attacker's cost, reflecting the typical cat-and-mouse dynamics in open package ecosystems [36].

4.2.1 Method. We randomly sampled 626 packages from the original 640,482 false positives reported by Neupane *et al.* [49], which have suspicious package names but are not flagged as malware in their respective security analyses. This sample size gives a confidence level of 99% with a margin of error of 5% on the distribution.

To identify possible metadata signals in the existing package confusion data, we began by having two researchers analyze 200 of these packages. They analyzed each package's metadata (*e.g.*, READMEs, maintainers, and version timelines). Each researcher independently proposed possible features based on this analysis (codebook) and then they discussed this codebook together to reach agreement on terms and definitions. To test validity, they then independently applied this codebook to the 200 packages and measured agreement using Cohen's Kappa [15]. The initial inter-rater reliability was 0.6 ("substantial" [45]). The researchers then convened

Table 3: Overview of the 15 metadata-based verification rules. Each rule lists its goal, the concrete implementation & *metadata* used (with inspected metadata fields *italicized*), and the mapped LLM directive. In the LLM Directive column, “—” denotes a rule implemented purely with metadata checks and no LLM prompt. The final four rules (R12–R15) were added during deployment to reduce false positives.

Rule	Description	Implementation & <i>metadata</i> used	LLM Directive
R1: Obviously Unconfusing	Determine if a package name conveys an intentional, brand-specific identity (e.g., <i>catplotlib</i>), clearly differentiating it from deceptive imitations.	Cross-reference flagged <i>package names</i> with known legitimate projects and use LLM analysis to assess whether the naming convention is deliberate and unambiguous for developers.	<i>obvious_not_typosquat</i>
R2: Distinct Purpose	Distinguish packages with different functionalities, even if names are superficially similar (e.g., <i>lodash-utils</i> vs. <i>lodash</i>).	Pass both <i>package descriptions</i> to the LLM; if the model reasons they serve genuinely different use cases or unique functionality, mark as distinct purpose, otherwise flag as possible impersonation.	<i>has_distinct_purpose</i>
R3: Fork Package	Detect benign forks sharing near-identical code or metadata with a popular package.	Compare <i>README files</i> , <i>version histories</i> , and <i>file structures</i> for high overlap. Similarities without malicious edits suggest harmless forks.	<i>is_fork</i>
R4: Actively Developed/Maintained	Determine if packages are frequently updated or actively maintained by multiple contributors, which are less likely to have malicious intent.	Retrieve <i>last-update timestamp</i> , <i>commit history</i> , and <i>version count</i> . Classify packages with recent updates (e.g., within 30 days) or more than five versions as legitimate.	—
R5: Comprehensive Metadata	Packages missing critical metadata elements are suspicious.	Check for <i>licenses</i> , <i>maintainer information</i> , and <i>repository links</i> .	<i>no_readme</i>
R6: Overlapped Maintainers	Distinguish legitimate extensions or rebrands by verifying if the flagged package shares maintainers with the legitimate one.	Match <i>maintainer identifiers</i> (e.g., email) between flagged and legitimate packages. Overlapping maintainers suggest legitimate intent.	<i>is_known_maintainer</i>
R7: Adversarial Package Name	Filter out name pairs with significant length differences, as these often indicate unrelated projects rather than covert mimicry.	Compare flagged and legitimate <i>package name lengths</i> . A difference exceeding 30% indicates likely unrelated naming.	<i>is_adversarial_name</i>
R8: Known Maintainers	Trust packages maintained by reputable/recognized authors/organizations.	Leverage knowledge in LLM training data to identify reputable <i>maintainers</i> .	<i>is_known_maintainer</i>
R9: Clear Description	Legitimate packages document purpose and usage.	Analyze repository metadata to ensure there are clear <i>package description</i> and <i>README</i> ; packages lacking such documentation should be flagged for further review.	<i>no_readme</i>
R10: Has Malicious Intent	Identify packages that deliberately mimic legitimate ones through near-identical descriptions, or have obviously suspicious content.	Use LLM to analyze the <i>package name</i> and <i>package description</i> to triage whether there is obvious malicious intent in the package.	<i>has_suspicious_intent</i>
R11: Test Package	Detect packages that are used for test or experiment purposes only.	Use LLM to analyze <i>package description</i> and determine whether it is for test.	<i>is_test</i>
R12: Package Relocation	Account for legitimate package relocations, common in hierarchical registries like Maven and Golang.	Parse <i>project configuration files</i> (e.g., look for <code><relocation></code> tag inside <code>pom.xml</code>). Identify and ignore renamed or migrated projects.	<i>is_relocated_package</i>
R13: Organization Allowed List	Prevent false positives by excluding packages published by trusted or verified organizations.	Maintain a <i>approved organizations list</i> . If a flagged package is published under an allowed organization (e.g., <code>@oxc-parser/binding-darwin-arm64</code>), it should be considered legitimate, comparing to <code>binding-darwin-arm64</code> .	—
R14: Domain Proxy/Mirror	Account for legitimate proxies or mirrors in the ecosystem, common in hierarchical registries like Golang.	Maintain a <i>recognized domains list</i> that serve as proxies or mirrors. If a given package is published under a valid domain (e.g., <code>gopkg.in/go-git/go-git</code>), consider it legitimate when compared to its primary source (<code>github.com/go-git/go-git</code>).	—
R15: Registered Prefix	Leverage NuGet’s prefix reservation: legitimate publishers can reserve name prefixes (e.g., <i>Newtonsoft.Json.*</i>).	NuGet offers a <i>verified</i> field to indicate if a package name has a registered prefix. A verified package is considered legitimate.	—

to resolve discrepancies and refine their analysis. Through discussion, they reached a consensus on measurable attributes that could indicate malicious intent or the possibility of a stealthy attack.

Based on the high agreement in this process, one of these researchers analyzed the remaining 426 packages.

4.2.2 Results. Of the 626 packages sampled, 601 were still accessible. Among these, we identified 480 benign packages (79.9%) and what we believe are 121 stealthy attacks (20.1%). The progression of our results and the manually labeled dataset are available in our artifact (§8). From this analysis, we distilled 11 package metadata-based rules that differentiate genuine confusion attacks from benign look-alikes. These attributes include factors such as a distinct purpose, adversarial package naming, and the comprehensiveness of available metadata. See Table 3 for ConfuGuard’s rules.

5 ConfuGuard Design and Implementation

We introduce ConfuGuard, our detector for package confusion threats and attacks. ConfuGuard is designed to meet the system requirements defined in §3.3. Leveraging the insights from our empirical study (§4), ConfuGuard integrates syntactic and semantic package name analysis, hierarchical naming checks, and verifications based on package metadata to enhance threat detection.

Figure 3 shows the components and parts of ConfuGuard:

Component I. Infrastructure:

- Part ①** A package metadata database, ensuring real-time awareness of new and evolving packages (**Req₄**).
- Part ②** Popularity metrics to protect high-value targets (**Req_{2,3}**).
- Part ③** A fine-tuned embedding model and database to capture domain-specific semantic name similarities (**Req_{1,3}**).

Component II. System Analysis:

Part ④ Syntactic and semantic confusion checks (**Req₂**).

Part ⑤ Filtering out benign packages using metadata (**Req_{1,4}**).

Component III. Human Analysis:

Part ⑥ Forwarding possible threats to security analysts (**Req_{1,4}**).

5.1 Part ①: Package Metadata Database

5.1.1 Rationale. Package confusion attacks are initiated regularly. Up-to-date metadata facilitates threat detection.

5.1.2 Approach. ConfuGuard relies on a database with package names, version histories, commit logs, license info, maintainer records, and other publicly accessible metadata, all updated on a weekly basis. Regular updates reduce concerns about stale data and ensure our results are up to date. We specifically use our industry partner’s private database, which consolidates metadata from NPM, PyPI, RubyGems, Maven, Golang, Hugging Face, and NuGet.

5.2 Part ②: Defining Trusted Resources

5.2.1 Rationale. Confusion attacks mimic trusted resources. If an attacker chooses a name that mimics an untrusted package, little harm can result. Untrusted packages can be compared against trusted ones to focus on the threats of highest potential impact.

5.2.2 Approach. Like prior works [49, 68], we operationalize trust in terms of popularity. Software engineers commonly make use of popularity signals as a proxy for trustworthiness [10], and thus attackers choose names similar to popular packages. Our specific popularity measure depends on the SPR. Many SPRs—in our case, NPM, PyPI, RubyGems, Hugging Face, and NuGet—offer weekly or monthly download counts. Our other two supported

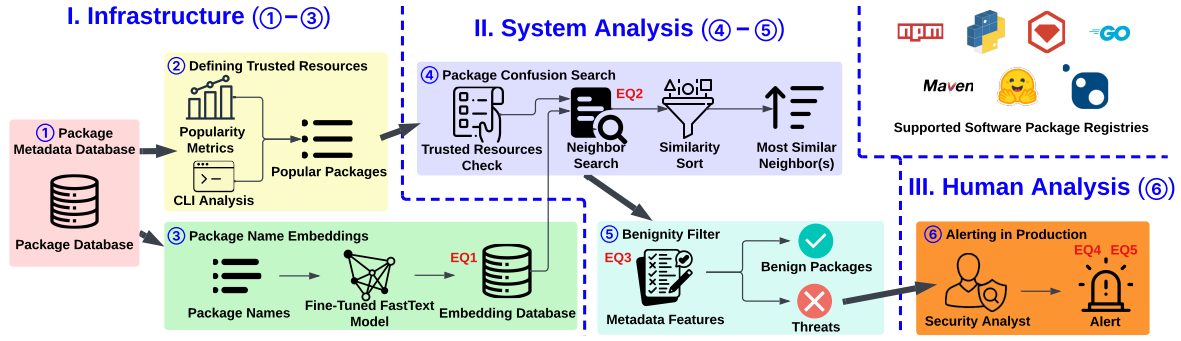


Figure 3: High-level architecture of ConfuGuard. There are three components divided into six parts. *Component I – Infrastructure (1–3)* builds the package-metadata database (§5.1), identifies trusted resources (§5.2), and constructs the package name embedding model (§5.3). *Component II – System Analysis (4–5)* performs the package-confusion search (§5.4) and benignity filtering (§5.5); these two parts are the main novelties of ConfuGuard. Components 1 and 2 together constitute the fully automated ConfuGuard pipeline. *Component III – Human Analysis (6)* routes possible package confusion attacks to security analysts for investigation (§5.6), whose decisions feed back into prompt, parameter, and threshold refinement to improve Components I and II. Red labels indicate where each evaluation question (EQ1–EQ5) is assessed.

SPRs, Maven and Golang, do not publish download metrics. For these, we make use of the `ecosyste.ms` database, which estimates popularity using indicators like stargazers, forks, and dependent repositories [48]. The thresholds were set based on production experience and costs. We initially chose 5,000 weekly downloads for NPM, PyPI, RubyGems, and NuGet; 1,000 for Hugging Face (normalized for its lower download rate [30, 31]); and an `avg_ranking` score of 10 for Maven and Golang. We then adjusted the Golang thresholds to 4 to balance production latency.

Adversaries may inflate the popularity statistics of their packages [28], which could cause a malicious (or decoy) package to become trusted. If our partner’s security analysts flag any package as suspicious (§5.6), it is removed from the list of trusted packages.

5.3 Part ③: Package Name Embeddings

5.3.1 Rationale. Detecting maliciously similar names requires knowledge of lexical, syntactic, and semantic variations. Simple Levenshtein edit distance methods omit domain-specific semantic nuances (e.g., `meta-llama` vs. `facebook-llama` on Hugging Face), while generic embedding models can introduce inaccuracies that result in higher false positive or false negative rates. Robust, fine-tuned embeddings address these shortcomings by providing enhanced semantic sensitivity, reducing erroneous alerts. Prior work applies embedding for word-wise semantic similarity checks [49]. We also use embedding for the whole name to support efficient neighbor search (§5.4). A generic embedding approach also supports various package naming conventions, e.g., differences by SPR.

5.3.2 Approach. An *embedding model fine-tuned on real package names* enhances semantic sensitivity, enabling the detection of adversarial or suspicious names. This capability is integral to assessing the risk associated with a package and serves as a cornerstone of our *intent-centric* approach. Following [49], we build upon FastText [9], starting with the pre-trained model pre-trained on `cc.en.300.bin`, and *fine-tune* it using all ~9.1 million package names in our package metadata database as of Nov. 2024 (§5.1). By fine-tuning, we expect to better capture domain-specific semantics.

We then use the fine-tuned embedding model to create an embedding database. Given a package name, we remove its delimiters

and compute the embedding of the concatenated string. For hierarchical names from Golang (domain/author/repository), Maven (`groupId:artifactId`), and Hugging Face (author/model), we split names to create distinct author and package identifier embeddings. This approach reduces false positives wherein similar packages from the same organization or author are incorrectly classified as confusing (§5.4).

The resulting embedding database occupies 24 GB. We use the vector format provided by `pgvector` due to its efficient vector operations for databases [53]. This setup facilitates rapid query-based lookups and supports subsequent steps in package neighbor searching. Our artifact has visualizations of the embedding vectors (§8).

5.4 Part ④: Package Confusion Search

Having obtained the appropriate infrastructure, we must now detect package confusion threats efficiently yet accurately. In this part, we focus on efficiently flagging possible package confusion threats.

5.4.1 Rationale. Modern SPRs are enormous, so any detection scheme must be designed with scalability in mind. Given the set of all packages in an ecosystem, we follow prior work [49, 68] by comparing the (relatively large) “long tail” of untrusted resources to the (relatively small) set of trusted resources.

5.4.2 Approach. To support the threshold choices and parameter settings used in ConfuGuard for detecting package confusion attacks, we studied the naming characteristics in these attacks. We analyzed the 1,239 legitimate and typosquatted package name pairs from NPM, PyPI, and RubyGems provided by Neupane *et al.* [49]. Our analysis examined two key aspects of legitimate and typosquatted package name similarities: lexical distance using Levenshtein distance and semantic similarity using cosine similarity. For semantic similarity measurements, we used cosine similarities of package names computed from our fine-tuned FastText embeddings (§5.3).

We confirmed that real-world package confusion attacks involve small syntactic changes and high semantic similarity. The median Levenshtein distance between typosquatted and legitimate package names is 1, while the mean distance is 2.54. Additionally, 77.7% of all package pairs exhibit a distance of 2 or less. Among the analyzed

package pairs, the mean cosine similarity was 0.95, and the median attained the maximum possible value of 1.

Trusted Resources Check: We compare untrusted resources (*i.e.*, unpopular packages) against all packages on the popular list. For trusted resources (*i.e.*, popular packages), we restrict comparisons to “more” trusted packages. A package is considered more trusted than a given trusted package if it has a download rate at least 10 times higher or an `ecosyste.ms` score at least twice as high—following the criteria previously used by our industry partner.

Neighbor Search (Package Name Similarity Search): We perform a distance search between each resource and the set of trusted resources. Syntactic similarity is measured using Levenshtein distance (threshold=2), seeded by our empirical study and refined during deployment. For semantic similarity, we apply a cosine similarity threshold of 0.93, likewise initialized from the empirical baseline and finalized through deployment feedback. As a special case for hierarchical names, we tighten the similarity thresholds to 0.99 for package names and 0.9 for author names to better detect “compound squatting” attacks (see Table 6); these tighter bounds reflect security analysts’ observations during our field tests.

To efficiently measure semantic similarity, we leverage the *Approximate Nearest Neighbors* method with the *HNSW* index [44] in PostgreSQL. We opted for *HNSW* over *IVFFlat* based on benchmarking results that show faster search speeds and minimal vector perturbation [70]. By partitioning the embedding space into multiple clusters, the *HNSW* index limits distance computations to a smaller subset of candidate packages, rather than exhaustively comparing all pairs (Req₂).

Similarity Sort and Most Similar Neighbor(s): Inspired by Neupane *et al.* [49], we define a similarity function that ranks neighbors based on Levenshtein distance, n-gram similarity, phonetics, substring matching, and fuzzy ratio. We treat the nearest neighbors as the most likely targets. The suspicious package and these nearest neighbors are propagated to the benignity check (§5.5).

In production, similarity sorting often returns multiple packages with near-perfect scores. §6.6 shows that adding more neighbors substantially increases latency and the cost of the benignity check, while delivering minimal accuracy gains. Based on our industry partner’s feedback, we restrict retrieval to the top-2 neighbors to optimize the trade-off between accuracy and speed.

5.5 Part ⑤: Benignity Filter

5.5.1 Rationale. Prior work used purely string-oriented methods, which often misclassify harmless or beneficial packages as confusion attacks. To mitigate false positives, we developed a metadata-driven benignity check inspired by a recent metadata-aware malware detector [66]. We seek to filter out legitimate packages based on explainable heuristics, avoiding unnecessary alerts.

5.5.2 Approach. Our analysis of false-positive data (§4.2) showed many reasons why legitimate packages may have names resembling other trusted resources. Initially, we iteratively developed 11 rules that utilize metadata features (*e.g.*, version history, maintainers) to distinguish package confusion attacks from benign packages. We apply these rules to assess the benignity of package neighbors (§5.4) in order to reduce the false positive rate. We added several additional rules during deployment. Table 3 summarizes the goals

and implementation for each rule. Once our name-based detector flags a package we retrieve its metadata and apply benignity check rules. A risk score is calculated using a simple weighted sum, which facilitated analyst feedback compared to a learned approach.

To realize this filter, we first distinguished the rules that can be assessed using Boolean algebra over package metadata (*e.g.*, R4), from those that require extensive analysis (*e.g.*, R2) or subjective reasoning (*e.g.*, R9). The former we implemented with conventional software, and for the latter we used an LLM. We tried two prompting approaches: rule-based and “LLM-as-judge” [25, 39]. For rule-based, we provided detailed rules imitating security analysts’ reasoning (*e.g.*, specific metadata they referenced). This detected known attacks (accuracy of 0.82 on NeupaneDB), but overfit, raising false positives on new data (accuracy of 0.53 on ConfuDB as of March 2025). We also tried the “LLM-as-a-Judge” pattern, casting the model as an evaluator guided by a structured decision framework but without excessive detail [25]. This pattern performed better, with accuracy of 0.83 on NeupaneDB and 0.68 on the latest ConfuDB (see Table 5).

```
# ----- system_prompt (excerpt) -----
You are an AI assistant helping to identify false positives in
potential typosquat packages
Analyze the given package names and descriptions to determine if a
package is a legitimate alternative or a malicious typosquat
DECISION FRAMEWORK
A. Package Name Analysis (is_adversarial_name):
- TRUE if: Names are confusingly similar
- FALSE if: Names are clearly distinct or branded
B. Legitimate Package Determination (obvious_not_typosquat):
- TRUE if: High edit distance AND distinct purpose AND good docs
- FALSE if: Low edit distance OR similar purpose OR poor docs
C. Package Categorisation Flags:
- is_fork: TRUE if: Package metadata explicitly states it is a fork
- has_distinct_purpose: TRUE if: Packages solve different tasks
- is_test: TRUE if: Package name/description mentions testing
# ----- user_prompt (excerpt) -----
PACKAGE COMPARISON ANALYSIS
Target Pkg.: typo_name; Target Pkg. Metadata: typo_metadata
Legit Pkg. Name: legit_name; Legit Pkg. Metadata: legit_metadata
Ecosystem: registry
Analyze the information above and answer the questions below using
TRUE or FALSE.
1. obvious_not_typosquat: Is 'typo_name' clearly NOT a typosquat?
2. is_adversarial_name: Can users confuse 'typo_name' with
   legit_name'?
3. is_fork: Is 'typo_name' a legitimate fork or variant?
...
9. is_relocated_package: Is this a known package relocation?
```

Listing 1: Excerpts of the system and user prompts used in the benignity filter. Full prompts are in our artifact (§8). We refined both prompts iteratively with feedback from security analysts, ensuring that the LLM’s analysis aligns with the rule set in Table 3.

Listing 1 shows the final system and user prompts. We use OpenAI’s o4-mini, which was the state-of-the-art model at the time.

5.6 Part ⑥: Alerting in Production

Use in Production: ConfuGuard is integrated into our industry partner’s security scanning system for software packages. They had previously relied primarily on content-based malware scanners, but these are too computationally costly to scale well. The partner’s former approach for first-pass detection of confusion attacks was a lexical check (Levenshtein distance) which had a high false positive rate. ConfuGuard replaced that approach. Analysts triage alerts

from ConfuGuard and the other scanners to make a threat assessment. If they deem a package to be malicious, that information is propagated to our partner’s customers via security feeds.

Experience-Driven Optimizations: During deployment we made several optimizations. We share three examples. *First*, we enhanced our metadata checks by analyzing false positives and adding rules R11–R15 to better distinguish benign packages (Table 3). *Second*, we improved accuracy by splitting long identifiers, such as Golang’s domain-based names, into their constituent parts and computing similarity on each part. *Third*, we found the LLM-as-Judge pattern was more effective than detailed prompts (§5.5).

6 Evaluation

To evaluate ConfuGuard, we pose five Evaluation Questions (EQs) to assess its performance at the component level (**EQ1–EQ3**) and the system level (**EQ4–EQ5**). At the component level, we measure the effectiveness of novel mechanisms introduced in ConfuGuard. At the system level, we evaluate its integrated functionality and scalability compared to baseline tools (§6.1), and ability to detect real-world package confusion threats. Additionally, we compare our approach to SOTA methods to benchmark its effectiveness. An overview of the evaluation process is illustrated in Figure 3.

Component-level: We assess how individual components contribute effectively to the overall system.

- **EQ1: Performance of Embedding Model.** What is the accuracy and efficiency of our embedding model? (§5.3)
- **EQ2: Neighbor Search Accuracy.** How effective is the neighbor search approach? (§5.4)
- **EQ3: Benignity Check Accuracy.** How much does the benignity filter reduce false positive rates? (§5.5)

System-level: We examine the performance of the full ConfuGuard system and compare to other approaches.

- **EQ4: Discovery.** Can ConfuGuard identify previously unknown package confusion threats?
- **EQ5: Baseline Comparison.** How does ConfuGuard perform in terms of accuracy and latency compared to SOTA tools?

All experiments run on a server with 32 CPU cores (Intel Xeon CPU @ 2.80GHz) and 256 GB of RAM. Notably, the training and fine-tuning of FastText models do not require GPUs.

6.1 Baseline and Evaluation Datasets

State-of-the-Art Baselines: We compare our system to the Levenshtein distance approach [75], and Typomind [49]. We consider OSSGadget [46] out of scope because it only handles lexical confusions and its latency for long package names makes it unsuitable for production. These are all state-of-the-art open-source tools.

Evaluation Datasets: We evaluate using two datasets:

- **NeupaneDB:** 1,840 packages from [49], including 1,239 confirmed real-world package confusion attacks, and 601 manually labeled data we analyzed in §4.2.
- **ConfuDB:** 2,361 packages triaged by security analysts, collected during the development and refinement of ConfuGuard (§5.6).

6.2 EQ1: Performance of Embedding Model

We measured effectiveness and efficiency of our embedding model.

6.2.1 Effectiveness of Embedding Model.

Method: We evaluate embedding-based similarity detection, comparing three approaches:

- (1) *Levenshtein Distance*, calculating the number of single-character edits required to change one package name to another.
- (2) *Pre-trained FastText* [9] (cc.en.300.bin), as used in the SOTA work Typomind to capture semantic relationships [49].
- (3) *Fine-tuned FastText (Ours)*, trained with a corpus of package names to better capture domain-specific similarities.

To systematically compare these methods, we construct a balanced test set consisting of both positive and negative pairs, with each category containing 1,239 data points [8, 19].

- **Positive Pairs:** All 1,239 of the real package confusions from NeupaneDB, which were confirmed and removed from SPRs [49].
- **Negative Pairs:** Created synthetically by randomly pairing unrelated package names from registries, ensuring low similarity scores across all tested methods. These pairs are guaranteed not to represent package confusion relationships.

For each approach, we applied a predefined similarity threshold to classify package pairs as potential package confusion attacks. The threshold was selected via a grid search over values from 0 to 1 in 0.01 increments to optimize Precision and Recall, ensuring effective identification of true confusions. Pairs with similarity scores above the threshold were classified as positive, while those below were classified as negative. False positives were subsequently filtered using our false-positive verification process (§5.5). We compare the Precision, Recall, and F1 scores for each approach.

Result: Table 4 presents the comparison between our fine-tuned model and baseline approaches. The results show that our approach outperforms both the Levenshtein Distance and Pretrained models, achieving the highest F1 score for positive pairs (0.95) while maintaining strong performance on negative pairs (0.98).

Table 4: Similarity effectiveness of using distance and embedding. We apply a grid search which automatically determines the optimal threshold to maximize F1 scores for positive pairs while maintaining relatively high F1 scores for negative pairs.

Model	Positive Pairs			Negative Pairs			Overall Score
	Prec	Rec	F1	Prec	Rec	F1	
Levenshtein Distance	1.00	0.80	0.89	1.00	1.00	1.00	0.94
Pretrained	1.00	0.85	0.92	1.00	0.98	0.99	0.96
Fine-Tuned (Ours)	1.00	0.90	0.95	1.00	0.96	0.98	0.96

We performed a grid search and analyzed the ROC curve to assess how the similarity threshold affects accuracy and to plot the ROC curve of our fine-tuned model. This analysis identified 0.93 as the optimal threshold, which we adopted in production. With an AUC of 0.94, the model demonstrates a balanced trade-off between precision and recall for both positive and negative pairs.

6.2.2 Efficiency of Embedding Model.

Method: To assess the efficiency of embedding database creation, we evaluated the cost of creating the embedding database (§5.3). We measured the throughput, latency, and overall overhead associated.

Table 5: Accuracy metrics for ConfuGuard and Typomind, detailing true/false positives and negatives, as well as precision, recall, F1 score, and accuracy for active, stealthy, and benign threats. Although Typomind flags all packages as suspicious, our benignity filter substantially reduces false positives in production. Due to availability of metadata, we evaluated ConfuGuard on 1,819 packages from NeupaneDB and 2,034 from ConfuDB. We note that ConfuGuard exhibits lower benign classification accuracy in ConfuDB because it flags more complex cases (e.g., compound squatting) as potential threats, a nuance that Typomind does not capture. ConfuDB data was collected between January and June 2025.

Dataset	Type	Pkgs	ConfuGuard (Ours)								Typomind							
			TP	FP	TN	FN	Prec	Rec	F1	Acc	TP	FP	TN	FN	Prec	Rec	F1	Acc
NeupaneDB	Active	1,239	1,103	0	0	131	1.0	0.89	0.94	0.89	1,000	0	0	239	1.0	0.80	0.89	0.81
	Stealthy	121	68	0	0	53	1.0	0.56	0.72	0.56	121	0	0	0	1.0	1.0	1.0	1.0
	Benign	480	0	117	347	0	–	–	–	0.72	0	480	0	0	–	–	–	0.00
	<i>Overall</i>	1,840	1,171	117	347	184	0.90	0.86	0.88	0.83	1,121	480	0	239	0.70	0.82	0.76	0.61
ConfuDB	Active	133	61	0	0	72	1.0	0.53	0.70	0.53	71	0	0	62	1.0	0.53	0.72	0.56
	Stealthy	409	238	0	0	171	1.0	0.74	0.85	0.73	219	0	0	190	1.0	0.64	0.78	0.64
	Benign	1,492	0	414	1,078	0	–	–	–	0.72	0	560	932	0	–	–	–	0.61
	<i>Overall</i>	2,034	299	414	1,078	243	0.42	0.55	0.48	0.68	290	560	932	252	0.28	0.62	0.39	0.62

Result: Creating the embedding database can be done in a reasonable amount of time, even for large SPRs. Total database preparation cost varied from 28–650 seconds (RubyGems, NPM), largely driven by the size of the dataset in each ecosystem. Throughputs ranged from ~4.9–13.6 K pkgs/sec (Maven, PyPI), with per-package latencies consistently below 0.2 ms. HNSW indexing adds < 8 seconds of upfront processing for each ecosystem.

6.3 EQ2: Neighbor Search Accuracy

Method: To evaluate neighbor search performance, we analyzed suspicious packages identified by Typomind. We use a threshold of 0.3 obtained from EQ1 (§6.2).

Result: Our neighbor search algorithm accurately detected 99% of the 1,239 real-world package confusion attacks from NeupaneDB [49]. Moreover, our method effectively flagged impersonation squatting attacks targeting hierarchical package names, notably identifying reported cases on Hugging Face [54] and Golang [11] that earlier methods overlooked. These findings confirm that our neighbor search algorithm achieves state-of-the-art performance.

6.4 EQ3: Benignity Check Accuracy

Method: Using the raw embedding output (i.e., packages flagged for name-based suspicion), we apply the benignity check from §5.5.

We first use cross-validation to learn the weights and measure false positive rates (FPR) using NeupaneDB. We then computed SHAP values [43] by learning the weighed sum through regression to quantify the importance of each metric defined in Table 2. The resulting SHAP plot provides clear insights into how these metrics contribute to our system’s decision-making process. Additionally, we use regression to learn the weights.

Result: Figure 4 shows the SHAP value plot which indicates that our rules used in the benignity filter are useful. The plot highlights that distinct purpose (R2) and suspicious intent (R10) are the most influential features driving the model’s predictions, suggesting that packages showing suspicious or unusual behavior strongly push the model toward a malicious classification. Conversely, features such as known maintainers (R8) and fork packages (R3) generally pull the model’s output toward benign, indicating they lower the likelihood of a threat. Meanwhile, no clear description also shows a notable positive effect, aligning with the idea that missing documentation can be an indicator of malicious intent (R9).

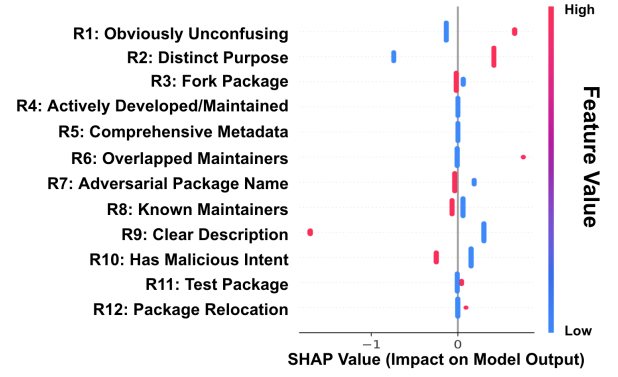


Figure 4: SHAP value plot for metadata verification over the learnable features from Table 3 (i.e., excluding R13, R14, R15). The x-axis shows each feature’s impact on model output, where positive values are towards benignity. Feature values are color-coded: red indicates high values with a strong, direct impact on the model output.

We use cross-validation to learn parameters and estimate the trade-off between reducing false positives and increasing false negatives. Our benignity filter correctly flagged 347 out of 480 (72%) false positives in the human labeled set. In the real-world attack set, it failed to classify 131 true positives (10%) out of the 1,234 that had available metadata. A detailed investigation revealed two main causes: (1) missing metadata, and (2) attacks that had been removed, with the package now maintained by npm, which the filter considers a valid maintainer.

These results confirm that by including supplementary heuristics beyond raw name similarity, false positive rates can be reduced while retaining high recall for genuinely malicious confusions.

6.5 EQ4: Discovery of New Package Confusions

Method: To evaluate the effectiveness of our package confusion detection system, we deployed ConfuGuard in a production environment for three month. During this period, flagged packages were analyzed using a commercial malware scanner and reviewed by threat analysts for detailed insights.

Result: Table 5 summarizes the package confusion attacks identified by ConfuGuard during the January–June 2025 deployment period (i.e., ConfuDB). Within this database, 1,686 packages were

labeled as false positives.³ Among the 630 remaining suspicious packages, 119 (19%) were identified as executable malware, 7 (1%) launched HTTP-flood or DDoS attacks, 6 (1%) loaded external scripts from unknown URLs or CDNs, and 1 (<1%) harvested analytics data and sent it to the attackers, and 497 (79%) carried out stealthy confusion attacks without obvious malicious content.

Table 6 shows the different kinds of package confusion threats identified by ConfuGuard during its 6 months of production use. ConfuGuard effectively detects advanced threats (§4.1)—including compound and impersonation squatting—across platforms such as Maven, Golang, and Hugging Face.

Table 6: ConfuGuard’s detection counts by confusion-attack category in production. This total exceeds the numbers in Table 5, as it reflects the cumulative counts collected over the six-month production deployment. Traditional lexical confusion types (targeted by Typomind and SpellBound) are listed first, then additional semantic confusions escaping prior work, and finally new patterns (see Table 2).

Attack Category	ConfuGuard	Typomind
Traditional Lexical Confusion	431	431
Additional Semantic Confusion	69	0
Impersonation Squatting	78	0
Compound Squatting	50	0
Domain Confusion	2	0
<i>Total</i>	630	431

We share four examples of the 630 attacks discovered by ConfuGuard and confirmed by analysts. (1) ConfuGuard identified an *impersonation squatting* attack on Maven: the malicious package `io.github.leetcrunch:scribejava-core` mimicked the legitimate `com.github.scribejava`. Although the package content appears identical at first glance, it contains code that injects a network call to steal user credentials. (2) ConfuGuard detected malware on a Golang package, where the username `boltdb-go` attempted to impersonate `boltdb`. ConfuGuard also flagged suspicious Hugging Face packages, such as `TheBlock/Mistral-7B-Instruct-v0.2-GGUF` (with only 217 downloads), mimicking benign package, `TheBloke/Mistral-7B-Instruct-v0.2-GGUF`, which has 92K monthly downloads. (3) ConfuGuard also captured *domain confusions*, such as Go packages from `github.phpd.cn` and `github.hscsec.cn` which look like proxies of official Golang packages but contain malware that either sets up a rogue MySQL server to steal sensitive files or intercepts and downgrades secure traffic to redirect data to attacker-controlled endpoints. (4) ConfuGuard uncovered a compound squatting attack on `@typescript-eslint/eslint-plugin`, where an attacker used a similar namespace and package identifier (`@typescript-eslint/eslint`) to deceive.

6.6 EQ5: System Accuracy and Efficiency

Method: We evaluated the end-to-end system accuracy and efficiency (*i.e.*, latency and throughput) by running ConfuGuard and Typomind on NeupaneDB and ConfuDB. We also performed a top- k analysis for ConfuGuard, varying the number of nearest neighbors in the benignity check to assess its impact on performance.

³The high false positive rate in ConfuDB is primarily due to the difficulty of confirming stealthy confusion attacks, which were left unreviewed by analysts.

Result: Table 5 presents the accuracy measurements. Across the datasets, ConfuGuard generally exhibits stronger performance in detecting active threats, achieving higher recall than Typomind while both maintain perfect precision. In the case of stealthy threats, the results are mixed: Typomind excels in the NeupaneDB dataset with a perfect recall, yet its relative performance declines on ConfuDB. For benign cases, the accuracy differs notably, as Typomind misclassifies benign packages in NeupaneDB, whereas it performs better on ConfuDB, though still lower than ConfuGuard’s performance in that category. Overall, ConfuGuard demonstrates strengths in detecting active and stealthy threats across datasets, reducing the FPR from 80% (§4.2) to 28% (133/480).

Our efficiency measurement shows that ConfuGuard incurs much higher latency (31s per package *vs.* 17s for Typomind) and correspondingly lower throughput (0.56 *vs.* 1.76 packages/s). However, in the broader deployment context, ConfuGuard significantly reduces the workload on analysts, resulting in notable end-to-end cost savings. The additional latency, stemming from benignity filtering and reliance on LLM-based benignity check, substantially enhances detection accuracy (reducing false positives).

Our top- k analysis shows that $k = 2$ offers the best balance. Moving from $k = 1$ to $k = 2$ raises ConfuGuard’s recall from 58% to 64%, accuracy from 54% to 58%, and F1 from 67% to 71%, while latency increases from about 10s to 14s. Increasing to $k = 3$ or 4 yields gains of less than 2 percentage points in each metric but pushes latency to 17s and 21s, respectively. Our partner’s separate malware scanner is expected to capture the remaining cases.

7 Discussion

7.1 Improvements on Prior Detectors

Having described ConfuGuard’s design in §5 and quantified its performance in §6, we now summarize ConfuGuard’s four key advances over previous detectors such as Typomind and SpellBound:

- (1) **Hierarchy-aware detection (§5.3).** By splitting each namespace component, ConfuGuard can detect compound and impersonation squats in hierarchical naming ecosystems (Table 2).
- (2) **Syntactic-Semantic Matching at Scale (§5.4).** ConfuGuard leverages a fine-tuned FastText model to identify potential package confusions with high recall and low latency, avoiding the exhaustive comparisons typical of prior tools (§6.2, §6.3).
- (3) **Metadata-driven benignity filter (§5.5).** ConfuGuard incorporates 15 heuristics based on package metadata to accurately identify package confusions. This technique reduces the false positive rate by 52%, compared to prior approaches (§6.4, §6.5).
- (4) **Engineered for production deployment (§5.6).** ConfuGuard detects package confusion threats at the scale required by modern software package repositories—which prior tools cannot achieve (§6.6). In practice, ConfuGuard reduces manual review effort and cost for security analysts by over 50%, so that even with higher per-package processing time, operational efficiency improves substantially.

7.2 Limitations

The primary design limitation of ConfuGuard is its reliance on software metrics to estimate whether a package is a confusion attack.

These metrics might be gamed. Little formal study exists, but recent work suggests both the possibility and real-world examples [28, 32].

A second design limitation is that the neighbor search relies on sub-word embeddings, which struggle to distinguish acronyms and short names (curse of dimensionality). *FastText* struggles with short words (e.g., `xml` vs. `yml` have a similarity score of 0.7). To mitigate attacks on short package names, we implemented a list of potential substitutions to identify cases of visual or phonetic ambiguity, and we combine embedding similarity with Levenshtein distance.

In our implementation, we rely on an LLM for benign filtering. This introduces correctness risks (hallucination, jailbreaking [78]). Adversaries might exploit this by tailoring their package name or metadata to persuade the LLM that the package is trustworthy using techniques like prompt injection or model hijacking [41, 81].

7.3 Lessons Learned from Production

We share four lessons learned over the six months of production deployment of ConfuGuard.

Lesson 1: ConfuGuard Prevents Real Confusion Attacks: Over the past six months, we deployed the system in our industry partner’s production environment, during which we identified and confirmed 630 threats, with 50,069 additional threats under review.

Lesson 2: Simplified LLM Prompts Unlock More Robust Reasoning: Our production experience revealed that overly detailed, rule-by-rule prompts can constrain an LLM’s ability to synthesize diverse signals, leading to both blind spots and bias [25]. For example, by instructing the model to treat any package with a namespace as “likely fork,” we missed malicious variants that reused legitimate namespaces but embedded suspicious payloads in their README or metadata. We replaced the myriad micro-rules with a decision framework approach (Listing 1), so the LLM can weigh name similarity, maintainer reputation, and other signals together.

Lesson 3: Ontology Matters: ConfuGuard is deployed as part of a security analysis pipeline, with human analysts to triage the results (§5.6). Analysts must interpret the results of ConfuGuard, and feel the package confusion categories outlined in Table 2 do not provide sufficient explanatory power. We propose to refine the confusion taxonomy to consider the malicious content and intent behind each package, including a risk-level classification.

Lesson 4: Large-Scale Empirical Feedback is Crucial: Our work with ConfuGuard and the experience-driven optimization pipeline (§5.6) underscores that only through large-scale empirical measurement can we systematically uncover the bottlenecks, failure modes, and hard edge cases that theory alone cannot predict. By continuously injecting human-in-the-loop analysis—where experts inspect anomalies and feed targeted insights back into the design—we close the gap between observed behavior and ideal performance by turning raw operational data into actionable system improvements. For example, a partner customer’s legitimate transitive package was falsely flagged as suspicious because it lacked a README. This led to the addition of an allow-list in ConfuGuard.

7.4 Future Directions

Enhancing Representations: Improving the representation of package names is crucial for more robust detection. While *FastText* captures semantic similarity through subword embeddings,

it struggles with typographical variations, particularly for short words or acronyms. In our experiments on Hugging Face packages, we observed uniformly high similarity scores due to long, acronym-dense names and extensive forking activity, which undermines standard confusion-detection methods [29]. LLMs can generate rich, domain-specific corpora, complete with synthetic typos and acronym variations. Applying (semi-)supervised learning to such tailored datasets can address these limitations.

One challenge based on our production experience is the limited availability of data for verifying package confusions. LLMs might help by augmenting training data with synthetic typos [71]. Advanced models, such as transformer-based architectures fine-tuned with contrastive learning on such typo-specific datasets, present a promising way for better detection accuracy and reliability. This approach has proven effective in combating domain typosquatting, but no research has been conducted targeting package confusions [35].

Human-in-the-Loop Optimizations: Our deployed system offers opportunities for further optimization, particularly through the incorporation of additional human-annotated ground truth. Drawing inspiration from human-in-the-loop reinforcement learning—where domain experts iteratively guide and refine agent behaviors—and self-adaptive prompting [57, 76], we propose integrating real-time feedback from security analysts into the reasoning processes of LLMs [24]. Analysts’ corrections and augmentations of intermediate model outputs serve as dynamic expert feedback, enabling improvements in both the prompts and the model’s behavior.

Anomaly Detection for Malicious Package Detection: Anomaly detection approaches offer significant potential for improving malicious package detection. By leveraging anomaly detection techniques and metadata analysis [27], future package confusion detectors might dynamically adapt to evolving attack strategies, e.g., learning new benignity rules without human oversight.

8 Conclusion

Modern software applications integrate third-party packages. Attackers therefore publish packages with names deliberately intended to confuse engineers, seeking to inject malware into production systems. Accurately detecting these attacks at ecosystem scale requires a low-latency system that accounts for human interpretations of package names. We showed that these requirements can be met by an embedding-based detection system combined with heuristics based on real-world attack patterns. Compared to the state-of-the-art, our ConfuGuard system captures additional confusion categories, achieves a substantially lower false-positive rate, and maintains acceptable latency for use either on registry backends or in handling on-demand requests. We conclude with insights from production experience and customer feedback.

Data Availability

Find our artifact at: <https://github.com/PurdueDualityLab/confuguard>. It includes everything but the commercial metadata database.

Acknowledgments

We acknowledge support from Socket. and from the National Science Foundation (NSF) awards #2135156, #2107230, and #2229703.

References

- [1] General Services Administration (GSA) 18F. 2025. 18F Open Source Policy. <https://18f.gsa.gov/open-source-policy/>.
- [2] Agency for Healthcare Research and Quality. 2019. Alert Fatigue. <https://psnet.ahrq.gov/primer/alert-fatigue>.
- [3] Sameer Ajmani. 2015. The Go Blog: Package Names. <https://go.dev/blog/package-names>.
- [4] Paschal C Amusuo, Kyle A Robinson, Tanmay Singla, Huiyun Peng, Aravind Machiry, Santiago Torres-Arias, Laurent Simon, and James C Davis. 2025. ZTD-JAVA: Mitigating Software Supply Chain Vulnerabilities via Zero-Trust Dependencies. In *Proceedings of the IEEE/ACM 47th International Conference on Software Engineering (ICSE'25)*.
- [5] Seth Art. [n. d.]. Whoami: A Cloud Image Name Confusion Attack. <https://securitylabs.datadoghq.com/articles/whoami-a-cloud-image-name-confusion-attack/>.
- [6] Hari Balakrishnan, Karthik Lakshminarayanan, Sylvia Ratnasamy, Scott Shenker, Ion Stoica, and Michael Walfish. 2004. A layered naming architecture for the internet. *ACM SIGCOMM Computer Communication Review* 34, 4 (2004), 343–352.
- [7] Alex Birsan. 2021. Dependency Confusion: How I Hacked Into Apple, Microsoft and Dozens of Other Companies. <https://medium.com/@alex.birsan/dependency-confusion-4a5d60fec610>.
- [8] Christopher M. Bishop. 2006. *Pattern Recognition and Machine Learning*. Springer, New York, NY.
- [9] Piotr Bojanowski, Edouard Grave, Armand Joulin, and Tomas Mikolov. 2017. Enriching Word Vectors with Subword Information. *Transactions of the Association for Computational Linguistics* 5 (2017), 135–146.
- [10] Hudson Borges and Marco Tulio Valente. 2018. What's in a GitHub Star? Understanding Repository Starring Practices in a Social Coding Platform. In *Journal of Systems and Software (JSS)*. <https://doi.org/10.1016/j.jss.2018.09.016>
- [11] Kirill Boychenko. 2025. Malicious Package Exploits Go Module Proxy Caching for Persistence. <https://socket.dev/blog/malicious-package-exploits-go-module-proxy-caching-for-persistence>.
- [12] Tohar Braun and Lidor Ben Shitrit. [n. d.]. Dependency Confusion: Supply Chain Attacks. <https://orca.security/resources/blog/dependency-confusion-supply-chain-attacks/>.
- [13] Benoit Bryon. 2012. PEP 423 – Naming conventions and recipes related to packaging. <https://peps.python.org/pep-0423/>.
- [14] Simon Butler, Michel Wermelinger, Yijun Yu, and Helen Sharp. 2010. Exploring the influence of identifier names on code quality: An empirical study. In *2010 14th European Conference on Software Maintenance and Reengineering*. IEEE, 156–165.
- [15] Jacob Cohen. 1960. A coefficient of agreement for nominal scales. *Educational and psychological measurement* 20, 1 (1960), 37–46.
- [16] NPM Contributors. 2025. NPM Package JSON: name. <https://docs.npmjs.com/cli/v9/configuring-npm/package-json#name>.
- [17] RubyGems Contributors. [n. d.]. RubyGems.org: Name your gem. <https://guides.rubygems.org/name-your-gem/>.
- [18] Conor Coyle. 2023. How Google Chrome Can Identify Typos in URLs to Keep You Safe Online. <https://www.makeuseof.com/google-chrome-identifies-url-typos/>
- [19] Richard O Duda, Peter E Hart, et al. 2006. *Pattern classification*. John Wiley & Sons.
- [20] Linda Erlenhov, Francisco Gomes De Oliveira Neto, and Philipp Leitner. 2020. An empirical study of bots in software development: Characteristics and challenges from a practitioner's perspective. In *Proceedings of the 28th ACM Joint Meeting on European Software Engineering Conference and Symposium on the Foundations of Software Engineering (ESEC/FSE'20)*. 445–455.
- [21] Gabriel Ferreira, Limin Jia, Joshua Sunshine, and Christian Kstner. 2021. Containing Malicious Package Updates in npm with a Lightweight Permission System. In *Proceedings of the IEEE/ACM 43rd International Conference on Software Engineering (ICSE'21)*. IEEE, 1334–1346. <https://doi.org/10.1109/ICSE43902.2021.00121>
- [22] The Apache Software Foundation. [n. d.]. Guide to Maven Naming Conventions. <https://maven.apache.org/guides/mini/guide-naming-conventions.html>.
- [23] Evgeniy Gabrilovich and Alex Gontmakher. 2002. The homograph attack. *Commun. ACM* (2002).
- [24] Marco Gerosa, Bianca Trinkenreich, Igor Steinmacher, and Anita Sarma. 2024. Can AI serve as a substitute for human subjects in software engineering research? *Automated Software Engineering* 31, 1 (2024), 13.
- [25] Jiawei Gu, Xuhui Jiang, Zhichao Shi, Hexiang Tan, Xuehao Zhai, Chengjin Xu, Wei Li, Yinghan Shen, Shengjie Ma, Honghao Liu, Saizhuo Wang, Kun Zhang, Yuanzhuo Wang, Wen Gao, Lionel Ni, and Jian Guo. 2025. A Survey on LLM-as-a-Judge. *arXiv preprint arXiv:2411.15594* (2025). <https://arxiv.org/abs/2411.15594>
- [26] Yacong Gu, Lingyun Ying, Yingyuan Pu, Xiao Hu, Huajun Chai, Ruimin Wang, Xing Gao, and Haixin Duan. 2023. Investigating package related security threats in software registries. In *2023 IEEE Symposium on Security and Privacy (SP)*. IEEE, 1578–1595.
- [27] Sajal Halder, Michael Bewong, Arash Mahboubi, Yinhao Jiang, Md Rafiqul Islam, Md Zahid Islam, Ryan HL Ip, Muhammad Ejaz Ahmed, Gowri Sankar Ramachandran, and Muhammad Ali Babar. 2024. Malicious Package Detection using Metadata Information. In *Proceedings of the ACM on Web Conference (WWW'24)*. 1779–1789.
- [28] Hao He, Haoqin Yang, Philipp Burckhardt, Alexandros Kapravelos, Bogdan Vasilescu, and Christian Kästner. 2026. Six Million (Suspected) Fake Stars on GitHub: A Growing Spiral of Popularity Contests, Spams, and Malware. In *Proceedings of the 48th International Conference on Software Engineering (ICSE'26)* (Rio de Janeiro).
- [29] Wenxin Jiang, Mingyu Kim, Chingwo Cheung, Heesoo Kim, George K Thiruvathukal, and James C Davis. 2025. "I see models being a whole other thing": an empirical study of pre-trained model naming conventions and a tool for enhancing naming consistency. *Empirical Software Engineering (EMSE)* 30, 6 (2025), 1–62.
- [30] Wenxin Jiang, Nicholas Synovic, Matt Hyatt, Taylor R. Schorlemmer, Rohan Sethi, Yung-Hsiang Lu, George K. Thiruvathukal, and James C. Davis. 2023. An Empirical Study of Pre-Trained Model Reuse in the Hugging Face Deep Learning Model Registry. In *Proceedings of the IEEE/ACM 45th International Conference on Software Engineering (ICSE'23)* (Melbourne, Australia).
- [31] Wenxin Jiang, Nicholas Synovic, Rohan Sethi, Aryan Indarapu, Matt Hyatt, Taylor R. Schorlemmer, George K. Thiruvathukal, and James C. Davis. 2022. An Empirical Study of Artifacts and Security Risks in the Pre-Trained Model Supply Chain. In *ACM Workshop on Software Supply Chain Offensive Research and Ecosystem Defenses (SCORED'22)* (Los Angeles, CA, USA), 105–114. <https://doi.org/10.1145/3560835.3564547>
- [32] Kelechi G Kalu, Sofia Okorafor, Betül Durak, Kim Laine, Radames C Moreno, Santiago Torres-Arias, and James C Davis. 2025. ARMS: A Vision for Actor Reputation Metric Systems in the Open-Source Software Supply Chain. *arXiv preprint arXiv:2505.18760* (2025).
- [33] Berkay Kaplan and Jingyu Qian. 2021. A survey on common threats in npm and pypi registries. In *International Workshop on Deployable Machine Learning for Security Defense*. Springer, 132–156.
- [34] Andreas D Kellas, Neophytos Christou, Wenxin Jiang, Penghui Li, Laurent Simon, Yaniv David, Vasileios P Kemerlis, James C Davis, and Junfeng Yang. 2025. PickleBall: Secure Deserialization of Pickle-based Machine Learning Models. In *Proceedings of the 2025 ACM SIGSAC Conference on Computer and Communications Security (CCS'25)*. 3341–3355.
- [35] Takashi Koide, Naoki Fukushima, Hiroki Nakano, and Daiki Chiba. 2023. PhishReplicant: A Language Model-based Approach to Detect Generated Squatting Domain Names. In *Proceedings of the 39th Annual Computer Security Applications Conference*. 1–13.
- [36] Piergiorgio Ladisa, Henrik Plate, Matias Martinez, and Olivier Barais. 2023. SoK: Taxonomy of Attacks on Open-Source Software Supply Chains. In *2023 IEEE Symposium on Security and Privacy (SP)*. 1509–1526. <https://doi.org/10.1109/SP46215.2023.10179304>
- [37] Jasmine Latendresse, Suhaib Mujahid, Diego Elias Costa, and Emad Shihab. 2022. Not all dependencies are equal: An empirical study on production dependencies in npm. In *Proceedings of the 37th IEEE/ACM International Conference on Automated Software Engineering (ASE)*.
- [38] Vladimir I. Levenshtein. 1966. Binary codes capable of correcting deletions, insertions, and reversals. *Soviet Physics Doklady* 10, 8 (1966), 707–710.
- [39] Dawei Li, Bohan Jiang, Liangjie Huang, Alimohammad Beigi, Chengshuai Zhao, Zhen Tan, Amrita Bhattacharjee, Yuxuan Jiang, Canyu Chen, Tianhao Wu, et al. 2025. From generation to judgment: Opportunities and challenges of llm-as-a-judge. In *Proceedings of the 2025 Conference on Empirical Methods in Natural Language Processing (EMNLP'25)*. 2757–2791.
- [40] Guannan Liu, Xing Gao, Haining Wang, and Kun Sun. 2022. Exploring the uncharted space of container registry typosquatting. In *31st USENIX Security Symposium (USENIX Security'22)*. 35–51.
- [41] Yi Liu, Gelei Deng, Yuekang Li, Kailong Wang, Zihao Wang, Xiaofeng Wang, Tianwei Zhang, Yepang Liu, Haoyu Wang, Yan Zheng, et al. 2023. Prompt Injection attack against LLM-integrated Applications. *arXiv:2306.05499* (2023).
- [42] JFrog Ltd. 2025. Hugging Face Naming Limitations. <https://jfrog.com/help/r/jfrog-artifactory-documentation/hugging-face-naming-limitations>.
- [43] Scott M Lundberg and Su-In Lee. 2017. A unified approach to interpreting model predictions. *Advances in neural information processing systems* (2017).
- [44] Yu A Malkov and Dmitry A Yashunin. 2018. Efficient and robust approximate nearest neighbor search using hierarchical navigable small world graphs. *IEEE transactions on pattern analysis and machine intelligence* 42, 4 (2018), 824–836.
- [45] Mary L McHugh. 2012. Interrater reliability: the kappa statistic. *Biochemia medica* 22, 3 (2012), 276–282.
- [46] Microsoft. [n. d.]. Microsoft OSSGadget. <https://github.com/microsoft/OSSGadget> et.
- [47] Muhammad Muzammil, Zhengyu Wu, Lalith Harisha, Brian Kondracki, and Nick Nikiforakis. 2024. Typosquatting 3.0: Characterizing Squatting in Blockchain Naming Systems. *arXiv:2411.00352* (2024).
- [48] Andrew Nesbitt. 2025. Ecosystems Database: A Comprehensive Dataset for Software Ecosystem Analysis. <https://ecosyste.ms/>

- [49] Shradha Neupane, Grant Holmes, Elizabeth Wyss, Drew Davidson, and Lorenzo De Carli. 2023. Beyond typosquatting: an in-depth look at package confusion. In *USENIX Security* '23. 3439–3456.
- [50] npm, Inc. 2024. Threats and Mitigations. <https://docs.npmjs.com/threats-and-mitigations#by-typosquatting--dependency-confusion>.
- [51] Marc Ohm and Charlene Stuke. 2023. Sok: Practical detection of software supply chain attacks. In *Proceedings of the 18th International Conference on Availability, Reliability and Security*. 1–11.
- [52] Chinenye Okafor, Taylor R. Schorlemmer, Santiago Torres-Arias, and James C. Davis. 2022. SoK: Analysis of Software Supply Chain Security by Establishing Secure Design Properties. In *Proceedings of the 2022 ACM Workshop on Software Supply Chain Offensive Research and Ecosystem Defenses*. ACM, Los Angeles CA USA, 15–24. <https://doi.org/10.1145/3560835.3564556>
- [53] pgvector Contributors. 2023. pgvector: A Vector Extension for PostgreSQL. <https://github.com/pgvector/pgvector>.
- [54] ProtectAI. n.d.. Unveiling AI Supply Chain Attacks on Hugging Face. <https://protectai.com/threat-research/unveiling-ai-supply-chain-attacks-on-hugging-face> Accessed: 2025-01-15.
- [55] Python Software Foundation. 2024. Acceptable Use Policy. <https://policies.python.org/pypi.org/Acceptable-Use-Policy/>.
- [56] Eric S. Raymond. 1999. *The cathedral & the bazaar: musings on Linux and open source by an accidental revolutionary* (1st ed ed.). O'Reilly, Beijing ; Cambridge, Mass.
- [57] Carl Orge Retzlaff, Srijita Das, Christabel Wayllace, Payam Mousavi, Mohammad Afshari, Tianpei Yang, Anna Saranti, Alessa Angerschmid, Matthew E Taylor, and Andreas Holzinger. 2024. Human-in-the-loop reinforcement learning: A survey and position on requirements, challenges, and opportunities. *Journal of Artificial Intelligence Research* 79 (2024), 359–415.
- [58] ReversingLabs. 2023. R77 Rootkit: Typosquatting Attack in NPM Ecosystem. <https://www.reversinglabs.com/blog/r77-rootkit-typosquatting-npm-threat-research>
- [59] Taylor R Schorlemmer, Kelechi G Kalu, Luke Chigges, Kyung Myung Ko, Eman Abu Ishgair, Saurabh Bagchi, Santiago Torres-Arias, and James C Davis. 2024. Signing in four public software package registries: Quantity, quality, and influencing factors. In *2024 IEEE Symposium on Security and Privacy (S&P'24)*. IEEE, 1160–1178.
- [60] Michael Scovetta. 2021. Improvements to oss-find-squats · Issue #226 · microsoft/OSSGadget. <https://github.com/microsoft/OSSGadget/issues/226>.
- [61] Lee Joon Sern and Yam Gui Peng David. 2021. TypoSwype: An imaging approach to detect typo-squatting. In *2021 11th IFIP International Conference on New Technologies, Mobility and Security (NTMS)*. IEEE, 1–5.
- [62] Ax Sharma. 2024. PyPI Crypto-Stealer Targets Windows Users, Revives Malware Campaign. *Sonatype Blog* (29 May 2024). <https://www.sonatype.com/blog/pypi-crypto-stealer-targets-windows-users-revives-malware-campaign>
- [63] César Soto-Valero, Amine Benelallam, Nicolas Harraud, Olivier Barais, and Benoit Baudry. 2019. The Emergence of Software Diversity in Maven Central. In *International Conference on Mining Software Repositories (MSR)*.
- [64] Stacklok. n.d.. Detecting Typosquatting Attacks on Open Source Packages Using Levenshtein Distance and Activity Data. <https://www.blackduck.com/resources/analyst-reports/open-source-security-risk-analysis.html>.
- [65] John Stenbit. 2003. Open source software (OSS) in the Department of Defense (DoD). *Department of Defense, Memorandum* (2003).
- [66] Xiaobing Sun, Xingan Gao, Sicong Cao, Lili Bo, Xiaoxue Wu, and Kaifeng Huang. 2024. 1 + 1 > 2: Integrating Deep Code Behaviors with Metadata Features for Malicious PyPI Package Detection. In *Proceedings of the 39th IEEE/ACM International Conference on Automated Software Engineering (ASE'24)*. 1159–1170.
- [67] Synopsys, Inc. 2024. 2024 Open Source Security and Risk Analysis Report. <https://www.blackduck.com/resources/analyst-reports/open-source-security-risk-analysis.html>.
- [68] Matthew Taylor, Ruturaj Vaidya, Drew Davidson, Lorenzo De Carli, and Vaibhav Rastogi. 2020. Defending against package typosquatting. In *Network and System Security: 14th International Conference (NSS)*.
- [69] Socket Research Team. [n. d.]. Malicious npm Package Typosquats Popular TypeScript ESLint Plugin. <https://socket.dev/blog/malicious-npm-package-typosquats-popular-typescript-eslint-plugin>.
- [70] Tembo. 2024. Vector Indexes in pgvector. <https://tembo.io/blog/vector-indexes-in-pgvector>.
- [71] Christopher Tozzi. 2024. Package Hallucination: The Latest, Greatest Software Supply Chain Security Threat? <https://blogs.idc.com/2024/04/22/package-hallucination-the-latest-greatest-software-supply-chain-security-threat/>. *IDC Blog* (April 22 2024).
- [72] Marcin Ulikowski. 2025. dnstwist: Domain name permutation engine for detecting homograph phishing attacks, typo squatting, and brand impersonation. <https://github.com/elceef/dnstwist>.
- [73] Nikos Vasilakis, Ben Karel, Nick Roessler, Nathan Dautenhahn, Andre DeHon, and Jonathan M. Smith. 2018. BreakApp: Automated, Flexible Application Compartmentalization. In *Proceedings 2018 Network and Distributed System Security Symposium*. Internet Society, San Diego, CA. <https://doi.org/10.14722/ndss.2018.23131>
- [74] Duc-Ly Vu, Fabio Massacci, Ivan Pashchenko, Henrik Plate, and Antonino Sabetta. 2021. LastPyMile: identifying the discrepancy between sources and packages. In *Proceedings of the 29th ACM Joint Meeting on European Software Engineering Conference and Symposium on the Foundations of Software Engineering*. ACM, Athens Greece, 780–792. <https://doi.org/10.1145/3468264.3468592>
- [75] Duc-Ly Vu, Ivan Pashchenko, Fabio Massacci, Henrik Plate, and Antonino Sabetta. 2020. Typosquatting and combosquatting attacks on the python ecosystem. In *EuroS&P Workshops*. IEEE, 509–514.
- [76] Xingchen Wan, Ruoxi Sun, Hanjun Dai, Serkan Arik, and Tomas Pfister. 2023. Better Zero-Shot Reasoning with Self-Adaptive Prompting. In *Findings of the Association for Computational Linguistics: ACL 2023*, Anna Rogers, Jordan Boyd-Graber, and Naoaki Okazaki (Eds.). Association for Computational Linguistics, Toronto, Canada, 3493–3514. <https://doi.org/10.18653/v1/2023.findings-acl.216>
- [77] Erik Wittern, Philippe Suter, and Shriram Rajagopalan. 2016. A look at the dynamics of the JavaScript package ecosystem. In *International Conference on Mining Software Repositories (MSR)*.
- [78] Sibo Yi, Yule Liu, Zhen Sun, Tianshuo Cong, Xinlei He, Jiaxing Song, Ke Xu, and Qi Li. 2024. Jailbreak Attacks and Defenses Against Large Language Models: A Survey. *arXiv preprint arXiv:2407.04295* (2024). <https://arxiv.org/abs/2407.04295>
- [79] Markus Zimmermann, Cristian-Alexandru Staicu, and Michael Pradel. 2019. Small World with High Risks: A Study of Security Threats in the npm Ecosystem. In *USENIX Security Symposium*. <https://doi.org/10.5555/3361338.3361407>
- [80] Tzachi Zornstein and Yehuda Gelb. 2023. A New, Stealthier Type of Typosquatting Attack Spotted Targeting NPM. *Checkmarx Blog* (9 May 2023). <https://checkmarx.com/blog/a-new-stealthier-type-of-typosquatting-attack-spotted-targeting-npm/>
- [81] Andy Zou, Zifan Wang, Nicholas Carlini, Milad Nasr, J. Zico Kolter, and Matt Fredrikson. 2023. Universal and Transferable Adversarial Attacks on Aligned Language Models. *arXiv preprint arXiv:2307.15043* (2023). <https://arxiv.org/abs/2307.15043>